StatsTracker Video Script

Hi, my name is Tyler Murray, and in this video I'm going to explain my StatsTracker class, how the code works, and how it interacts with AnalyticsManager.

The main purpose of StatsTracker is to take raw gameplay data that has already been collected by AnalyticsManager and turn it into a clean dashboard-friendly summary. AnalyticsManager is responsible for storing the logs, while StatsTracker is responsible for reading those logs and building summaries that the GUI can display.

In StatsTracker.hpp, there are three important structs. The first is StatSummary, which represents one numeric statistic, such as damage dealt or enemies killed. It stores a stable key, a display label, the current value, min, max, mean, median, and the number of samples.

The second struct is ActionSummary. This is used for action-log data instead of numeric data. It stores the key, label, total action count, and the most active entity if there is one.

The third struct is DashboardSnapshot. This is the final object that StatsTracker produces. It contains a vector of numeric stats and a vector of action stats. This makes it easier for the GUI to render numeric data and action data separately.

The first main function is BuildSeriesSummary. This function takes in a key, a label, and a DataLog. If the log has no samples, it returns std::nullopt, meaning there is no summary to display. If the log does contain data, it creates a StatSummary and fills in the sample count, minimum, maximum, mean, median, and current value. The current value comes from the most recent sample in the log.

The second main function is BuildActionSummary. This works similarly, but for an ActionLog. If there are no actions, it returns std::nullopt. Otherwise, it creates an ActionSummary with the total number of actions and the most active entity.

The main function that ties everything together is BuildSnapshot. This function takes a const reference to an AnalyticsManager. It asks the analytics manager for the enemies killed log, the damage dealt log, and the action log. Then it calls BuildSeriesSummary or BuildActionSummary for each one. If a summary exists, it gets added to the DashboardSnapshot.

This shows the relationship between the two classes. AnalyticsManager owns the actual data logs. It has methods like LogDamageDealt, LogEnemiesKilled, and LogAction, and it provides read-only access to those logs through getters. StatsTracker does not own or modify the data. It just reads from AnalyticsManager and formats that data into a summary.

There is also a DisplayData function in StatsTracker.cpp. This prints the dashboard snapshot to the console. It is mostly useful as a temporary or debugging display while the GUI version is being developed. The more important long-term path is BuildSnapshot, because that returns structured data that other parts of the program can use.

Overall, StatsTracker acts as the bridge between raw analytics data and display-ready dashboard data. AnalyticsManager collects and stores the data, and StatsTracker turns that data into summaries that are easier for the GUI and the player to understand.